

# Towards In-Situ Neuromorphic Computing Architecture for Event Stream Super-Resolution

Yihe Yu\*, Bo Li\*, Kexin Huang, Wei Liu, Jinghai Wang, Yue Liu, Zhiyi Yu, Shanlin Xiao†  
Sun Yat-sen University, China

{yuyh25, libo93, huangkx36, wangjh335, liuy2389}@mail2.sysu.edu.cn; {liuw359, yuzhiyi, xiaoshlin}@mail.sysu.edu.cn

**Abstract**—Event-based cameras, with their unique event stream representation, effectively mitigate motion blur in high-speed, high-exposure scenarios but suffer from low spatial resolution. To address this, we propose a super-resolution hardware accelerator for event streams based on Spiking Neural Networks (SNNs). In terms of network architecture, we incorporate hardware-friendly algorithmic designs by simplifying neuron models and optimizing convolution operations. On the hardware side, the design adopts a hierarchical structure featuring a highly parallel computational array. Additionally, by proposing a Kernel-Channel-Timestamp-Row (KCTR) dataflow and dual-pipeline structure, the design achieves in-situ computing, eliminating intermediate storage within layers and significantly reducing inter-layer spike storage. Evaluations on the N-MNIST and ASL-DVS datasets demonstrate root mean square errors (RMSE) of 1.296 and 0.121 for reconstructed super-resolution event streams. In downstream applications, the classification accuracies reach 98.84% and 99.73%, respectively. The proposed accelerator, designed using a 28nm CMOS process, improves reconstruction speed by 95.6% compared to a GPU, operates at 500 MHz, and consumes only 0.546 pJ per synaptic operation.

**Index Terms**—Spiking Neural Network, Hardware Accelerator, Super Resolution, Event-based Camera.

## I. INTRODUCTION

The event camera is a bio-inspired sensor that fundamentally differs from traditional cameras, which capture full scene frames at fixed intervals. Instead, event cameras, also known as Dynamic Vision Sensors (DVS) [1]–[4], respond asynchronously to brightness changes, outputting data as an event stream. When the light intensity at a pixel changes significantly beyond a predefined threshold, the pixel generates an event, indicating brightness increase or decrease (1 or 0). Unlike traditional cameras, event cameras eliminate frame-based redundancy, producing pixel-level event streams only during scene changes. Fig. 1 compares the imaging performance of two types of cameras for high-speed object. The event camera, which records only events and features higher temporal resolution compared to traditional frame-based cameras, effectively mitigates the issue of motion blur.

Event cameras offer distinct advantages over traditional frame-based cameras. They achieve microsecond-level temporal resolution [5], [6], far surpassing conventional frame rates,

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62334014; in part by the Key-Area Research and Development Program of Guangdong Province under Grant 2023B0303030004.

\* These authors contributed equally to this work.

† Corresponding author.

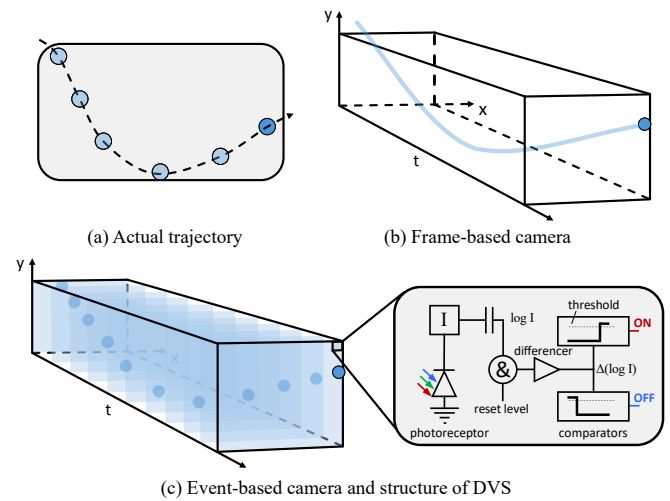


Fig. 1. (a) for the actual trajectory of object motion, (b) (c) illustrate the imaging results of a frame-based camera and an event-based camera for high-speed moving objects, and (c) shows the internal structure of DVS.

and their low latency effectively reduces motion blur. With independently and asynchronously operating pixels, they provide a dynamic range of up to 140 dB, compared to 60 dB in traditional cameras [2]. Additionally, their sparse data output, generated only during brightness changes, eliminates frame-based redundancy, streamlining processing and improving computational efficiency. These features significantly reduce computational and energy demands, making event cameras ideal for edge computing and real-time applications [7].

However, the spatial resolution of current event cameras remains lower than that of traditional cameras. Popular models like the DVS128 (128×128 pixels) [8] and DAVIS240 (240×180 pixels) [1] operate at relatively low resolutions due to their pixel-level event-triggering mechanism, where each pixel independently detects light intensity changes and generates events. As shown in Fig. 1(c), the DVS architecture integrates circuits like photoreceptors, differencers, and threshold comparators within each pixel [9], requiring substantial physical space and limiting resolution. Moreover, event cameras prioritize high-temporal-resolution data for compact edge devices, making it difficult to increase resolution within size constraints. Additionally, manufacturing costs and process limitations further restrict their spatial resolution [10].

Researchers have thus focused on super-resolution reconstruction of low-spatial-resolution images captured by event

cameras. The event streams generated by event cameras are asynchronous, sparse, and feature high temporal resolution, aligning well with the processing mechanisms of SNNs. As neural networks designed to handle discrete time-stamped data [11], SNNs can simultaneously process spatiotemporal information, making them particularly suitable for receiving and processing discrete event streams from event cameras [12]–[14]. However, current research on super-resolution for event cameras has largely focused on algorithms, with limited emphasis on hardware implementation [15]–[18]. To the best of our knowledge, this work is the first to integrate SNNs with event stream super-resolution within a algorithm-hardware co-design framework. We present an SNN-based hardware accelerator that generates high-resolution (HR) event streams from low-resolution (LR) inputs and adapts to diverse datasets. Our main contributions are as follows:

- We propose a hardware-efficient SNNs algorithm for event stream super-resolution image reconstruction. By simplifying the neuron model and optimizing computation order and channel count to exploit hardware parallelism.
- We propose a high-efficiency SNNs hardware architecture with hierarchical design. With the help of the Kernel-Channel-Timestamp-Row (KCTR) dataflow and a dual-pipeline strategy, in-situ computation is achieved, eliminating intermediate value storage within layers and reducing spike storage between layers.
- The performance of hardware-based super-resolution image reconstruction and the quality of the reconstructed images in downstream applications were systematically evaluated. Results show the proposed method reduces reconstruction error and significantly enhances downstream classification accuracy, confirming the broad potential of super-resolution in practical use.

## II. BACKGROUND

### A. Spiking neuron model

Spiking neurons are the core of SNNs [19], with models like the Hodgkin-Huxley [20], Leaky Integrate-and-Fire (LIF) [21], and Izhikevich [22] proposed to mimic biological neurons. The LIF model, favored for its simplicity and low computational cost, is the most widely used but lacks accuracy for complex tasks [23]. To address this, the Spike Response Model (SRM) [24] improves on LIF by incorporating refractory response (REF) decay, boosting accuracy with minimal overhead [25], [26]. Thus, we adopt the SRM in our SNNs design.

Fig. 2 shows the dynamics of an SRM spiking neuron. Input spikes  $s_{in}(t)$ , weighted by synaptic strengths (indicated by spike lengths), are processed sequentially to modulate the post-synaptic potential (PSP). The membrane potential  $u(t)$  (red curve) combines all PSPs and the REF (blue curve). When  $u(t)$  exceeds the threshold  $V_{th}$ , the neuron emits an output spike  $s_{out}(t)$ , triggering a REF that temporarily suppresses the membrane potential, preventing premature firing and ensuring proper spike timing.

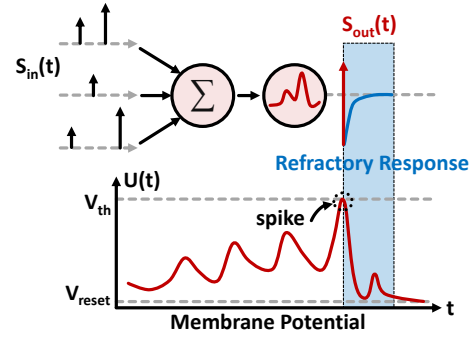


Fig. 2. Membrane potential dynamics of the SRM neuron. The red curve represents the neuronal membrane potential  $U(t)$ , which is the sum of all PSPs and the REF (blue curve).

The formula for calculating the membrane potential of SRM neurons is as follows [16]:

$$u(t) = (\gamma(t) * s_{out}(t)) + \sum_i w^i (\epsilon(t) * s_{in}^i(t)) \quad (1)$$

In this equation,  $s_{in}(t)$  and  $s_{out}(t)$  denote the input and output spikes of the SRM neuron, respectively. The input spike train for the  $i$ -th synapse is given by  $s_{in}(t) = \sum_k \delta(t - t_k^i)$ , where  $t_k^i$  represents the timestamp of the  $k$ -th spike in this sequence. Each input spike train is associated with a corresponding weight  $w^i$ . The membrane potential of the neuron,  $u(t)$ , generates an output spike when it exceeds the threshold voltage  $V_{th}$ . As shown in Equation 1,  $u(t)$  is defined as the sum of all PSPs and REFs. Specifically, the PSP is obtained by convolving the input spike train  $s_{in}(t)$  with the spike response kernel  $\epsilon(t)$  and multiplying the result by the corresponding synaptic weight  $w^i$ . Similarly, the REF is derived by convolving the output spike train  $s_{out}(t)$  with the REF kernel  $\gamma(t)$  [27]. The mathematical definitions of the spike response kernel and the REF kernel are as follows, where  $\tau_s$  and  $\tau_r$  are the time constants of the respective kernels, and  $\Theta(t)$  denotes the Heaviside step function.

$$\epsilon(t) = \frac{t}{\tau_s} e^{1-\frac{t}{\tau_s}} \Theta(t) \quad (2)$$

$$\gamma(t) = -2V_{th} e^{-\frac{t}{\tau_r}} \Theta(t) \quad (3)$$

### B. Principle of event cameras

The event stream output of an event camera is a three-dimensional dataset  $M(x, y, t)$ , containing spatial coordinates and timestamps. At time  $t$ , the event stream output at coordinate  $(x, y)$  can be expressed as [28]:

$$M(x, y, t) = \begin{cases} 1, & \int_{t-\Delta t}^t E(t) dt > \phi, \\ 0, & \int_{t-\Delta t}^t E(t) dt < \phi \end{cases} \quad (4)$$

The photoreceptor in an event camera converts brightness changes into an electrical signal  $E(t)$ , which is integrated and compared to a threshold  $\phi$ . A spike is emitted, and the integrator resets to zero when the accumulated signal exceeds  $\phi$ , indicating a detectable brightness change. If the threshold is not reached, the signal continues accumulating. In the model,

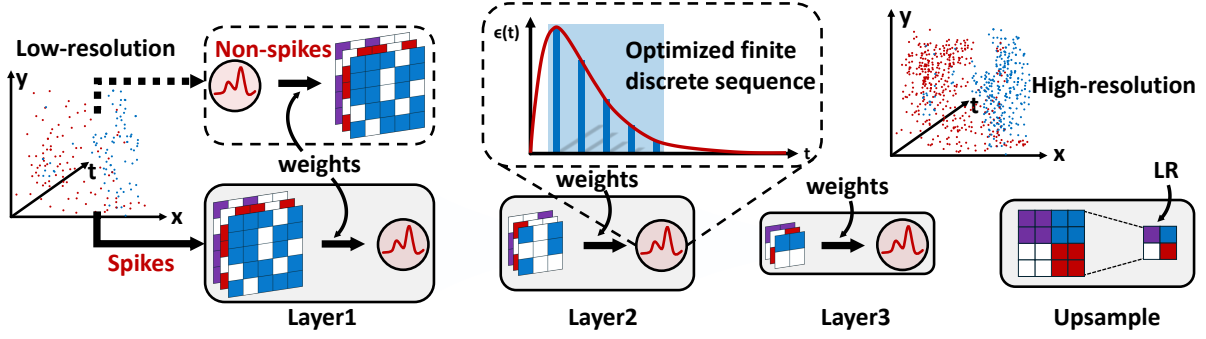


Fig. 3. The overall network architecture for event stream super-resolution. The LR event stream passes from left to right through two convolution layers, one deconvolution layer, and three neuron layers, then combines with the output of the upsampling layer to ultimately produce the HR event stream.

a “1” represents a spike at pixel  $(x, y, t)$ , generating an event, while “0” indicates no event [29].

The left plot in Fig. 3 shows the discrete event stream output of the event camera, where red and blue pixels represent positive and negative polarities, corresponding to brightness increases and decreases. Event camera data contains spatial and temporal information along the  $x$ ,  $y$ , and  $t$  axes, abstracted as a 3D space for processing. Positive and negative polarity events are separated into two input streams, processed independently by the SNNs.

### III. EVENT STREAM SUPER-RESOLUTION SNNs ALGORITHM

To achieve efficient algorithm-hardware co-design, we propose a hardware-friendly SNNs model for event stream super-resolution image reconstruction. The overall network architecture is illustrated in Fig. 3. The following sections provide a detailed description of its components.

#### A. Overall architecture

Our network model includes three components: a convolutional module for 2D spatial processing, a neuronal module for 1D temporal processing, and an interpolation module for upsampling. The convolutional module consists of two convolutional layers and one deconvolutional layer, forming a convolutional spiking neural network. The first layer uses a  $5 \times 5$  kernel, stride 1, and 2-pixel padding; the second uses a  $3 \times 3$  kernel, stride 1, and 1-pixel padding. The deconvolutional layer has a  $2 \times 2$  kernel, stride 1, and no padding. Both input and output event streams have 2 channels, while internal channels are set to  $n = 4$ . The input and output channel dimensions for the three-layer network are  $2n$ .

The neuron module uses SRM neurons as the network core. The input is a low-resolution event stream from an event camera, with 2 channels representing positive and negative polarity events, corresponding to pixel brightening and darkening. The SRM’s membrane potential is calculated using Equation 1, with time constants for the response kernels and REF kernels set to  $\tau_s = \tau_r = 1, 2, 4$  across the network’s three levels.

The interpolation module enhances spatial details in LR images to reconstruct a high-resolution (HR) event stream,

providing higher-quality input for subsequent tasks. Two convolutional layers extract features from the LR event stream without increasing spatial resolution. The deconvolutional layer enlarges feature maps via transposed convolution, while the upsampling module restores spatial details using interpolation. The outputs of these components are summed to generate the final HR event stream, achieving a resolution four times higher than the original.

#### B. Optimization of the spatial convolution

One advantage of SNNs is their use of discrete spikes as input data [30], enabling convolution’s multiply-accumulate operations to be reduced to simple accumulation, which significantly decreases computational complexity.

As illustrated in the dashed box on the left of Fig. 3, the original network architecture applies a 1D convolution (accumulation) with the spike response kernel in the temporal dimension, followed by a 2D convolution (multiply-accumulate) with the spatial kernel. To further simplify the 2D convolution, we reordered these operations, performing 2D convolution before 1D convolution. This adjustment reduces the complex 2D convolution to simple accumulation, effectively lowering the network’s computational complexity. For the three-layer structure, the neuron computation in layer  $l$  is formulated as:

$$\mathbf{u}^{(l+1)}(t) = \epsilon(t)(\mathbf{W}^{(l)} * \mathbf{s}^{(l)}(t)) + (\gamma(t) * \mathbf{s}^{(l+1)}(t)) \quad (5)$$

$$\mathbf{s}^{(l+1)}(t) = \sum_{t_k \in \{t | \mathbf{u}^{(l+1)}(t) = V_{th}\}} \delta(t - t_k) \quad (6)$$

#### C. Optimization of the temporal neuron

In SRM neuron model, the spike response and REF kernels are exponential functions, which are challenging to implement in hardware. To address this, we discretized these functions by sampling them at discrete time points  $t = 0, 1, 2, \dots$ , converting them into 1D convolution sequences [31].

Given that the response kernel decays rapidly after  $t = 1$  and falls below 0.01 for  $t > 6$ , values with absolute magnitudes below 0.01 were discarded, further reducing the kernel to a compact finite sequence. This method transformed the spike response and REF kernels in all three network layers into discrete, finite 1D convolution sequences. This method

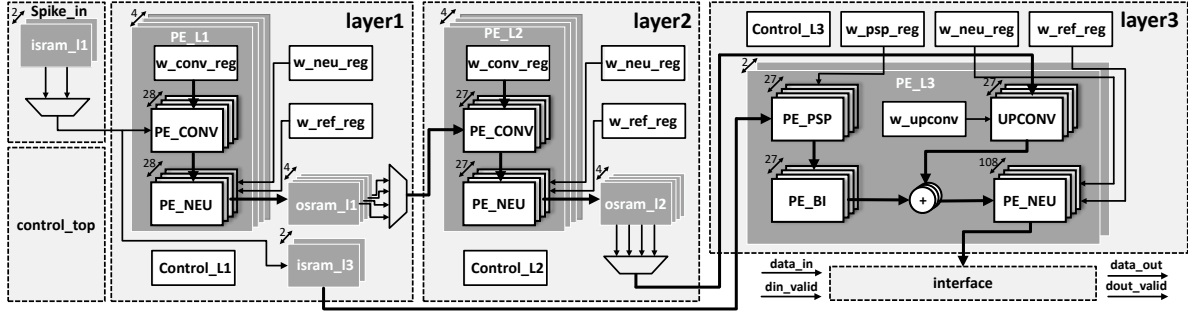


Fig. 4. The overall architecture of the hardware accelerator for event stream super-resolution. The hardware architecture consists of three hierarchical levels to enable highly parallel computation, employing a combination of KCTR data flow and dual-pipeline design to achieve in-situ computing.

not only eliminates the need for complex exponential computations in hardware implementation but also preserves the original network's performance to the greatest extent possible.

#### D. Optimization of the upsampling interpolation

Common upsampling methods, such as bilinear and nearest-neighbor interpolation, achieve similar model performance. Nearest-neighbor interpolation directly maps the original image to the new image, minimizing computational overhead [32]. Additionally, it reads each row of data only once, significantly reducing memory access frequency.

Therefore, we choose nearest-neighbor interpolation for the upsampling layer, while the floating-point operations in the model are quantized to fixed-point representations. Model weights are scaled by 4 using shift operations and rounded to integers in the range  $[-128, 127]$ , representing signed 8-bit hardware data. Consequently, the outputs of both 2D and 1D convolutions are scaled down by right-shifting 2 bits. Similarly, the discrete 1D convolution sequences for the spike and REF kernels are quantized like the weights, with their convolution results also right-shifted by 2 bits.

### IV. EVENT STREAM SUPER-RESOLUTION HARDWARE

#### A. Hierarchical hardware architecture

The overall hardware architecture of the SNNs accelerator is shown in Fig. 4, with the dashed box representing the first hierarchy. This includes the top-level control module (control\_top), interface module (interface), input spike module (spike\_in), and three computational layers (layer1-3).

The control\_top module oversees initialization, process control, and termination. The interface module handles signal transmission across clock domains. The spike\_in module, comprising two SRAMs, stores input spikes from two channels. Layers 1-3 implement the computational logic of the neural network's three layers.

The second hierarchy, shown as overlapping gray boxes, includes identical multi-channel processing modules (PE\_L1, PE\_L2, PE\_L3), spike storage modules, and control units with weight registers for each layer. Input spikes are processed row by row through 2D convolution units, with results passed to neuron computation units to calculate PSP and REF, generating output spikes stored in output memory (OSRAM). To enable parallel processing across four output channels, each

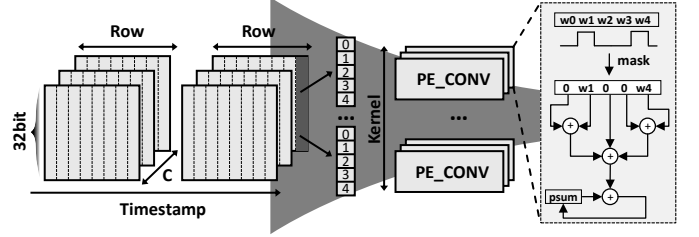


Fig. 5. Kernel-Channel-Timestamp-Row dataflow.

channel uses separate 2D convolution kernel weights, requiring each PE\_L1 (or PE\_L2) to have its own convolution weight register (w\_conv\_reg). In contrast, the 1D convolution kernel weights for PSP and REF calculations are shared across all four channels, so all PE\_L1 (or PE\_L2) modules use common 1D kernel registers (w\_neu\_reg and w\_ref\_reg).

The second hierarchy's internal structure comprises parallel processing units, forming the third hierarchy. Its input spikes come partly from the second hierarchy's output spikes and partly from the initial input spikes temporarily stored in the first hierarchy. The processing flow is as follows: output spikes from the second hierarchy are processed by the deconvolution unit (PE\_UPCONV), while initial input spikes pass through the 1D convolution unit (PE\_CONV) and the interpolation unit (PE\_BI). The results from both paths are summed and sent to the neuron unit (PE\_NEU), which computes PSP and REF to generate the final output spikes.

#### B. Key technologies of in-situ computing

In our SNNs hardware architecture, traversing convolution kernels, channels, timestamps, and matrix rows involves extensive computations and frequent memory access to update membrane potentials and spike weights, leading to increased latency and energy consumption [33]. To address this, we optimized the dataflow and proposed a novel KCTR dataflow with a dual-pipeline control structure. This design maximizes neuron reuse, enhances parallelism, and reduces memory data migration. By eliminating intermediate value storage within layers and minimizing inter-layer spike storage, it significantly improves hardware efficiency.

1) *KCTR Dataflow*: As shown in Fig. 5, the implementation process of the KCTR dataflow is demonstrated. Input spikes are 32 bits wide, and image data is segmented into 32-bit groups with zero-padding for incomplete parts. The KCTR



dataflow transmits data sequentially in the order of Kernel, Channel, Timestamp, and Row.

During computation, the 32-bit input spike is distributed across four output channels for separate processing. Within each channel, it is grouped based on the convolution kernel size  $k$  and stride  $s$ . In the first layer, the input spike is divided into 28 groups of 5-bit sequences using a sliding stride of  $s = 1$ . These sequences are transmitted to 28 parallel 2D convolution units, while the weight register sends 5 corresponding 2D convolution weights to enable parallel processing.

In a single 2D convolution unit, input spikes are masked, multiplied by weights, and accumulated, with the results stored in the 2D convolution result register (psum). These operations align strictly along the four dimensions: Kernel, Channel, Timestamp, and Row. Each computation unit processes one row of input spikes per clock cycle, while generating a set of output spikes involves  $kernel\_size \times input\_channels$  computations.

The top-level control module uses multiple counters to manage layer-wise computations sequentially. Each layer's control module comprises four counters: kernel (cnt\_kernel), channel (cnt\_channel), timestamp (cnt\_timestamp), and output row (cnt\_row), which form the core of the KCTR dataflow control. These counters ensure orderly data transmission and task execution within each layer. Under the KCTR framework, precise synchronization between the top-level and layer control modules is achieved through counter increments, enabling accurate data flow, efficient pipelining, and parallel computation.

The first-layer convolution unit processes input data across four dimensions: Kernel, Channel, Timestamp, and Row. The kernel counter spans 0–4, the channel counter 0–1, the timestamp counter 0– $T$  (where  $T$  is the dataset's number of timestamps), and the output row counter depends on the matrix size. The input spike memory's read address is dynamically computed based on these counters, enabling row-wise reading of five input rows per channel. The control module sequentially activates the 2D convolution unit, accumulating results over 10 iterations in the 2D convolution result register. Once a row of output is generated, the psum data is transferred to the neuron computation unit, and the register is reset for the next cycle.

2) *Dual-Pipeline*: As shown in Fig. 6, we designed a dual-pipeline architecture for the SNNs hardware accelerator that, combined with the KCTR dataflow, enables high parallelism and in-situ data processing, reducing storage overhead. Within each layer, a MicroPipeline coordinates the parallel operation of computing units for efficient task execution. Between layers, a MacroPipeline optimizes computational efficiency, this allows concurrent operation of the three hardware modules, greatly enhancing overall performance.

The MicroPipeline operates with a 12-clock-cycle period, enabling all computational units to complete a full process within each cycle. During single-layer computations, tasks collaborate in a pipeline: as the 2D convolution unit transfers a partial sum (psum) to the neuron unit, neuron computation starts immediately while the convolution unit begins process-

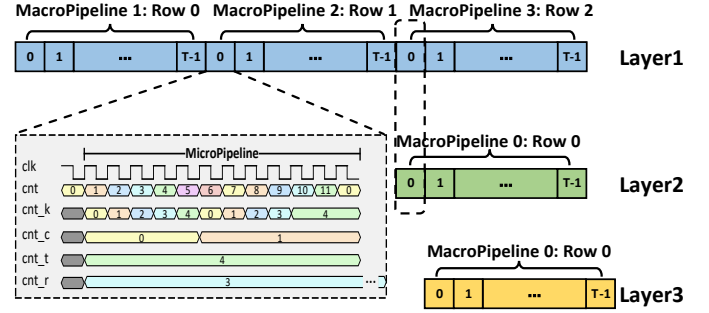


Fig. 6. The MicroPipeline, responsible for intra-layer control, and the MacroPipeline, responsible for inter-layer control.

ing the next row. Within a single task, stages like address generation, spike reading, and psum accumulation also execute in a pipelined manner.

In the hardware architecture, module states are governed solely by the MicroPipeline counter and enable pulse signals. In the first layer, the KCTR counters in the control module are tightly synchronized with the top-level MicroPipeline counter, with both increment and reset operations driven by it. When the counter reaches 11, pvalid is asserted in the next cycle, signaling the completion of 2D convolution and triggering the neuron computation unit. All related signals propagate seamlessly across stages according to computation clock cycles, eliminating the need for additional control logic and ensuring efficient coordination.

The MacroPipeline facilitates efficient inter-layer pipelining and minimizes inter-layer spike storage by integrating closely with the MicroPipeline. Output spikes from each layer are stored row by row in SRAM, with all  $T$  timestamps for a row saved before proceeding to the next. Computation is also row-wise, processing  $T$  timestamps sequentially per row. Thus, the MacroPipeline cycle spans  $T$  MicroPipeline cycles, requiring  $T \times 12$  clock cycles in total.

After three MacroPipeline cycles, the first layer generates output spikes for the first three rows, each with  $T$  timestamps. In the following MicroPipeline cycle, the first layer processes the 0th timestamp of the 3rd row, while the second layer reads the 0th timestamp spikes from the 0th, 1st, and 2nd rows of the first layer's output to compute the 0th timestamp of the 0th row for the third layer.

In the next MicroPipeline cycle, the first layer processes the 1st timestamp of the 3rd row. Simultaneously, the second layer reads the 1st timestamp spikes from the first layer's output and computes the 1st timestamp of the 1st row for the third layer. Meanwhile, the third layer reads the 0th timestamp spike of the 1st row from the second layer's output to compute the final output spikes for the 0th and 1st rows, both for the 0th timestamp. This design synchronizes inter-layer computations, minimizing storage for intermediate spike outputs.

## V. EXPERIMENTS

### A. Performance evaluation of super-resolution reconstruction

To evaluate the super-resolution image reconstruction accuracy of the designed SNNs hardware accelerator, we conducted

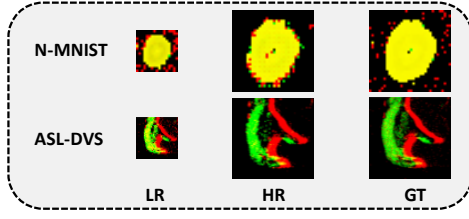


Fig. 7. Reconstruction results of images from different datasets.

TABLE I

COMPREHENSIVE EVALUATION OF RECONSTRUCTION PERFORMANCE.

|                |         | Neurocomputing [15] |          | ICCV [16]      | This work |
|----------------|---------|---------------------|----------|----------------|-----------|
| Implementation |         | GPU                 | GPU      | ASIC           |           |
| RMSE           | N-MNIST | 0.757               | 0.272    | 1.296          |           |
|                | ASL-DVS | 0.229               | 0.179    | <b>0.121</b>   |           |
| Acc            | N-MNIST | 97.80%              | 99.70%   | 98.84%         |           |
|                | ASL-DVS | 98.00%              | 99.90%   | 99.73%         |           |
| Speed          | N-MNIST | Na                  | 38.99ms  | <b>1.69ms</b>  |           |
|                | ASL-DVS | Na                  | 164.39ms | <b>22.37ms</b> |           |
| Power          |         | Na                  | 30W      | <b>649mW</b>   |           |

hardware-based tests using the spike-based N-MNIST [34] and ASL-DVS [35] datasets. The N-MNIST dataset comprises 70,000 handwritten digit images (0–9) captured from the MNIST dataset using a moving sensor, while the ASL-DVS dataset includes 100,800 gesture samples of 26 English letters (a–z) recorded in real-world scenarios.

Fig. 7 shows super-resolution results for two datasets: a handwritten “0” and the hand gesture for “c.” From left to right are the low-resolution input (LR), super-resolved output (HR), and ground truth (GT). In both cases, the HR closely matches the GT, clearly identifying the targets. Notably, HR images also exhibit reduced edge noise and improved sharpness compared to GT.

Currently, hardware implementations of SNNs for event stream super-resolution image reconstruction remain unexplored. To fill this gap, we compare our hardware-based SNNs with prior software-based ones. Table I summarizes metrics including RMSE, task accuracy, speed, and energy efficiency. While software SNNs (typically GPU-based with floating-point precision) offer higher accuracy, they require much more time and energy.

Compared to studies [15] and [16], the proposed design achieves lower reconstruction error on the ASL-DVS dataset but slightly higher error on the N-MNIST dataset, primarily due to the nearest-neighbor interpolation method, which is more error-prone on datasets with smaller sample sizes.

The SNNs hardware accelerator achieves 98.84% and 99.73% accuracy on N-MNIST and ASL-DVS datasets, respectively, outperforming software-based SNNs in [15] and trailing [16] by less than 1%. It consumes only 664mW, much lower than the 30W of GPU-based implementations. This peak power, measured at max frequency, can be reduced by lowering frequency, enabling flexible performance-power trade-offs ideal for energy-sensitive edge applications.

Processing times are also markedly improved: 1.69 ms per image for N-MNIST (vs. 38.99 ms on GPU) and 22.37 ms for

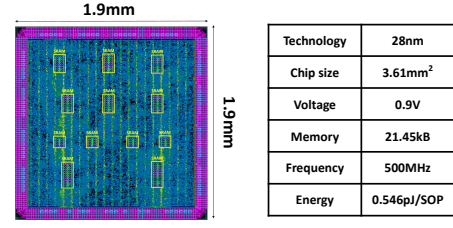


Fig. 8. Hardware accelerator layout and key technical parameters.

TABLE II

COMPARED WITH STATE-OF-THE-ART NEUROMORPHIC ACCELERATORS.

|                                   | VLSI [36]      | ISSCC [37]  | JSSC [38]   | TCAS-I [39]    | This work          |
|-----------------------------------|----------------|-------------|-------------|----------------|--------------------|
| Technology                        | 40nm           | 28nm        | 28nm        | 40nm           | 28nm               |
| Area                              | 0.84*0.84      | 0.67*0.67   | 0.93*1.77   | 0.95*0.92      | 1.9*1.9            |
| Neurons                           | Na             | 256-256-16  | 1024-512-10 | Na             | 289-2312-4624-9248 |
| Neuron density/(mm <sup>2</sup> ) | Na             | 748         | 949         | Na             | <b>4563</b>        |
| Memory(kB)                        | 53.84          | 138         | 266.5       | 130.25         | <b>21.45</b>       |
| Weight bitwidth                   | 8/16bit        | 8bit        | 8bit        | 8/16bit        | 8bit               |
| Frequency (MHz)                   | 259            | 115         | 210         | 150            | <b>500</b>         |
| Energy(pJ/SOP)                    | 1.89@0.65V     | 5.3@0.5V    | 1.5@0.56V   | 9.9@0.75V      | <b>0.546@0.9V</b>  |
| Application                       | Image          | Gesture     | Keyword     | Image          | Event stream       |
|                                   | Classification | Recognition | Recognition | Classification | Super-resolution   |

ASL-DVS (vs. 164.39 ms), representing speed gains of 86.4% and 95.6%, respectively.

#### B. Performance evaluation of the SNNs hardware accelerator

The proposed hardware accelerator, implemented using a 28nm CMOS process, as shown in Fig. 8. We compare it with SNNs hardware accelerators designed for other applications, as no dedicated SNNs hardware exists for image super-resolution reconstruction. Table II summarizes the results.

In terms of hardware resources, larger SNNs with more layers and neurons enhance parallelism and speed but consume more hardware resources. We implement the largest network scale within a moderate chip area, achieving higher neuron density than [36]–[39]. Memory usage is minimized via the KCTR dataflow and dual-pipeline design, which avoid storing intermediate values and reduce inter-layer spike storage.

The SNNs hardware accelerator designed in this study operates at a maximum frequency of 500 MHz, achieving computational efficiency far exceeding that of studies [36]–[39], while maintaining exceptionally low power consumption. Measured by average energy per SOP, the proposed hardware achieves an energy efficiency of 0.546 pJ/SOP under high-frequency, highly parallel operation, reducing energy consumption by 64% to 90% compared to other SNNs accelerators.

## VI. CONCLUSION

Event cameras use event stream representation to effectively reduce motion blur and overexposure but are limited by their low spatial resolution. To address this, We propose the first event stream super-resolution hardware accelerator utilizing SNNs. The architecture features hardware-friendly algorithms via neuron model simplification and convolution optimization. A hierarchical design enables highly parallel computation, utilizing KCTR dataflow and dual-pipeline mechanisms for in-situ processing, which eliminates intermediate value storage and minimizes inter-layer spike storage.

## REFERENCES

- [1] C. Brandli, R. Berner, M. Yang, S.-C. Liu, and T. Delbruck, "A  $240 \times 180$  130 db 3  $\mu$ s latency global shutter spatiotemporal vision sensor," *IEEE Journal of Solid-State Circuits*, vol. 49, no. 10, pp. 2333–2341, 2014.
- [2] G. Gallego, T. Delbrück, G. Orchard, C. Bartolozzi, B. Taba, A. Censi, S. Leutenegger, A. J. Davison, J. Conradt, K. Daniilidis *et al.*, "Event-based vision: A survey," *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 1, pp. 154–180, 2020.
- [3] X. Zheng, Y. Liu, Y. Lu, T. Hua, T. Pan, W. Zhang, D. Tao, and L. Wang, "Deep learning for event-based vision: A comprehensive survey and benchmarks," *arXiv preprint arXiv:2302.08890*, 2023.
- [4] T. Delbruck and M. Lang, "Robotic goalie with 3 ms reaction time at 4% CPU load using event-based dynamic vision sensor," *Frontiers in neuroscience*, vol. 7, p. 223, 2013.
- [5] H. Kim, S. Leutenegger, and A. J. Davison, "Real-time 3D reconstruction and 6-DoF tracking with an event camera," in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part VI 14*. Springer, 2016, pp. 349–364.
- [6] M. Bouvier, A. Valentian, and G. Sicard, "Scalable Pitch-Constrained Neural Processing Unit for 3D Integration with Event-Based Imagers," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 385–390.
- [7] A. Amir, B. Taba, D. Berg, T. Melano, J. McKinstry, C. Di Nolfo, T. Nayak, A. Andreopoulos, G. Garreau, M. Mendoza *et al.*, "A low power, fully event-based gesture recognition system," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 7243–7252.
- [8] P. Lichtsteiner, C. Posch, and T. Delbruck, "A  $128 \times 128$  120 dB 15  $\mu$ s Latency Asynchronous Temporal Contrast Vision Sensor," *IEEE Journal of Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, 2008.
- [9] C. Posch, D. Matolin, and R. Wohlgenannt, "A QVGA 143 dB dynamic range frame-free PWM image sensor with lossless pixel-level video compression and time-domain CDS," *IEEE Journal of Solid-State Circuits*, vol. 46, no. 1, pp. 259–275, 2010.
- [10] B. Son, Y. Suh, S. Kim, H. Jung, J.-S. Kim, C. Shin, K. Park, K. Lee, J. Park, J. Woo *et al.*, "A  $640 \times 480$  dynamic vision sensor with a 9 m pixel and 300meps address-event representation," 2017.
- [11] P.-Y. Chuang, P.-Y. Tan, C.-W. Wu, and J.-M. Lu, "A 90nm 103.14 TOPS/W Binary-Weight Spiking Neural Network CMOS ASIC for Real-Time Object Classification," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [12] Y. Cao, Y. Chen, and D. Khosla, "Spiking deep convolutional neural networks for energy-efficient object recognition," *International Journal of Computer Vision*, vol. 113, pp. 54–66, 2015.
- [13] D. Gehrig, M. Gehrig, J. Hidalgo-Carrió, and D. Scaramuzza, "Video to events: Recycling video datasets for event cameras," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 3586–3595.
- [14] D. Sharma, A. K. Kosta, and K. Roy, "Lightning Talk: A Perspective on Neuromorphic Computing," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–2.
- [15] H. Li, G. Li, and L. Shi, "Super-resolution of spatiotemporal event-stream image," *Neurocomputing*, vol. 335, pp. 206–214, 2019.
- [16] S. Li, Y. Feng, Y. Li, Y. Jiang, C. Zou, and Y. Gao, "Event stream super-resolution via spatiotemporal constraint learning," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 4480–4489.
- [17] X. Xiang, L. Zhu, J. Li, Y. Wang, T. Huang, and Y. Tian, "Learning super-resolution reconstruction for high temporal resolution spike stream," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 33, no. 1, pp. 16–29, 2021.
- [18] A. I. Maqueda, A. Loquercio, G. Gallego, N. García, and D. Scaramuzza, "Event-based vision meets deep learning on steering prediction for self-driving cars," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 5419–5427.
- [19] G. Wulfram and M. K. Werner, "Spiking neuron models: Single neurons, populations, plasticity," *Encyclopedia Neurosci*, vol. 4, pp. 277–280, 2002.
- [20] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *The Journal of physiology*, vol. 117, no. 4, p. 500, 1952.
- [21] W. Gerstner and W. M. Kistler, *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge university press, 2002.
- [22] E. M. Izhikevich, "Simple model of spiking neurons," *IEEE Transactions on neural networks*, vol. 14, no. 6, pp. 1569–1572, 2003.
- [23] P. Dayan and L. F. Abbott, *Theoretical neuroscience: computational and mathematical modeling of neural systems*. MIT press, 2005.
- [24] Gerstner, "Statistical physics, plasmas, fluids, and related interdisciplinary topics," *American Physical Society. (2000). Physical review. E*, 1995.
- [25] R. Jolivet, T. J. Lewis, and W. Gerstner, "Generalized integrate-and-fire models of neuronal activity approximate spike trains of a detailed model to a high degree of accuracy," *Journal of neurophysiology*, vol. 92, no. 2, pp. 959–976, 2004.
- [26] H. Fang, B. Taylor, Z. Li, Z. Mei, H. H. Li, and Q. Qiu, "Neuromorphic Algorithm-hardware Codesign for Temporal Pattern Learning," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 361–366.
- [27] W. M. Kistler, W. Gerstner, and J. L. van Hemmen, "Reduction of the Hodgkin-Huxley equations to a single-variable threshold model," *Neural Computation*, vol. 9, no. 5, pp. 1015–1045, 1997.
- [28] T. Serrano-Gotarredona and B. Linares-Barranco, "A  $128 \times 128$  1.5% Contrast Sensitivity 0.9% FPN 3  $\mu$ s Latency 4 mW Asynchronous Frame-Free Dynamic Vision Sensor Using Transimpedance Preamplifiers," *IEEE Journal of Solid-State Circuits*, vol. 48, no. 3, pp. 827–838, 2013.
- [29] P. Bardow, A. J. Davison, and S. Leutenegger, "Simultaneous optical flow and intensity estimation from an event camera," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 884–892.
- [30] M. Pfeiffer and T. Pfeil, "Deep learning with spiking neurons: opportunities and challenges," *Frontiers in neuroscience*, vol. 12, p. 409662, 2018.
- [31] D. R. Kunkle and C. Merrigan, "Pulsed neural networks and their application," *Computer Science Dept., College of Computing and Information Sciences, Rochester Institute of Technology*, 2002.
- [32] C. Dong, C. C. Loy, K. He, and X. Tang, "Image super-resolution using deep convolutional networks," *IEEE transactions on pattern analysis and machine intelligence*, vol. 38, no. 2, pp. 295–307, 2015.
- [33] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.
- [34] G. Orchard, A. Jayawant, G. K. Cohen, and N. Thakor, "Converting static image datasets to spiking neuromorphic datasets using saccades," *Frontiers in neuroscience*, vol. 9, p. 437, 2015.
- [35] Y. Bi, A. Chadha, A. Abbas, E. Bourtsoulatzé, and Y. Andreopoulos, "Graph-based object classification for neuromorphic vision sensing," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 491–501.
- [36] P.-Y. Tan and C.-W. Wu, "A 40-nm 1.89-pJ/SOP Scalable Convolutional Spiking Neural Network Learning Core With On-Chip Spatiotemporal Back-Propagation," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2023.
- [37] C. Frenkel and G. Indiveri, "ReckOn: A 28nm sub-mm<sup>2</sup> task-agnostic spiking recurrent neural network processor enabling on-chip learning over second-long timescales," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
- [38] J. Zhang, D. Huo, J. Zhang, C. Qian, Q. Liu, L. Pan, Z. Wang, N. Qiao, K.-T. Tang, and H. Chen, "ANP-I: A 28-nm 1.5-pJ/SOP Asynchronous Spiking Neural Network Processor Enabling Sub-0.1-  $\mu$ J/Sample On-Chip Learning for Edge-AI Applications," *IEEE Journal of Solid-State Circuits*, 2024.
- [39] C. Liu, Z. Yang, X. Zhang, Z. Zhu, H. Chu, Y. Huan, L.-R. Zheng, and Z. Zou, "A Low-Power Hybrid-Precision Neuromorphic Processor With INT8 Inference and INT16 Online Learning in 40-nm CMOS," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 10, pp. 4028–4039, 2023.